

Constraint Annotation Patterns for Trustworthy AI-Assisted Prototype Generation

Anonymous Authors

Abstract. In Domain-Driven Design (DDD), domain models capture business knowledge, while OCL constraints precisely specify business rules. Current DDD methods employ annotation-based Domain-Specific Languages (aDSLs) to encode domain models using annotations that define their structure and behavior. However, such aDSLs often fall short in capturing and integrating complex OCL constraints. This paper improves the expressiveness of aDSLs by introducing *Constraint Annotation Patterns* (CAPs), a reusable specification mechanism for representing complex domain constraints at the domain-model level. By deriving OCL invariants through a deterministic generation mapping and validating them before code generation, CAPs also provide a principled foundation for trustworthy AI-assisted prototype generation, in which AI-suggested constraint specifications are formally verified before entering the generation pipeline. We define a catalog of CAPs, where each CAP consists of a *Class Diagram Structure*, a *Parameterized OCL Template*, and an *Annotation Specification*. Concrete constraints are instantiated by binding the parameters defined in the Annotation Specification to elements of Domain Class Specification Language (DCSL) models, from which OCL invariants are automatically derived for validation and subsequent model-driven processing. A supporting tool, CAP/DCSL, extends the UML-based Specification Environment (USE) with CAP specification and automatic OCL derivation. Application to three heterogeneous case studies illustrates that CAP covers all recurring expressiveness criteria identified across the case studies while significantly reducing manual validation code compared with existing annotation-based approaches.

Keywords: Domain-Driven Design · Domain-Specific Language · UML/OCL · Large Language Models · Trustworthy AI.

1 Introduction

Domain-Driven Design (DDD) [13] places the problem domain and business logic at the center of software development. Its core artifact is the domain model, which captures business knowledge, while OCL constraints [23] formally specify business rules. An effective domain model should be *feasible*, supporting automated model-to-code transformation, and *satisfiable*, accurately representing business requirements and constraints. Constructing such models—particularly specifying their constraints precisely—remains a demanding manual task, for which large language models (LLMs) are increasingly being explored as assistants [18, 8]. However, LLM-generated specifications are stochastic and error-prone [2] and therefore cannot be incorporated into a model-driven generation pipeline without systematic verification. Trustworthy AI-assisted prototype generation thus requires a specification mechanism that both constrains and verifies AI-suggested constraints.

Domain models [30, 25, 19, 4, 15] are commonly represented using three complementary approaches. First, UML/OCL class diagrams [24] provide a standard representation of domain concepts and constraints, but a semantic gap remains between conceptual models and executable implementations. Second, external DSLs [14, 3, 31, 9] offer dedicated syntaxes for domain modeling and automation, but require specialized toolchains for parsing and code generation. Recent approaches such as B-OCL [17] improve constraint specification, yet executable code generation and runtime OCL validation remain challenging. Third, annotation-based internal DSLs [5, 6] integrate domain specifications directly into host languages such as Java [29], providing a lightweight implementation-oriented solution. However, existing approaches [25, 19] support only limited subsets of OCL and lack reusable abstractions for recurring complex domain constraints.

Previous studies such as DCSL [20] and AGL [11] employ an annotation-based DDD approach (aDSL) to construct executable domain models covering both structural and behavioral aspects. Annotations are embedded directly in the source code, tightly connecting domain model design with its implementation and enabling the automatic generation of software artifacts from the domain model. However, these aDSLs fall short in capturing complex OCL constraints, which are essential for precisely specifying business rules in DDD [13, 23], and in composing domain concerns within an executable unified domain model for automatic prototype generation, as introduced in UDML [22].

These limitations motivate the need for a reusable annotation-based specification mechanism that can systematically represent complex OCL constraints while maintaining the simplicity and practicality of annotation-based DSLs—a mechanism that, as argued above, is also the prerequisite for trustworthy AI-assisted generation.

In this paper, we introduce *Constraint Annotation Patterns* (CAPs), a reusable specification mechanism for representing recurring complex OCL constraints in annotation-based domain models. CAPs enable constraints to be specified declaratively, automatically derived as OCL invariants, and integrated into a *Unified Domain Model* (UDM) for model-driven processing. The approach is evaluated on three heterogeneous case studies (*CourseMan*, *OrderMan*, and *ProcessMan*) with respect to expressiveness and coding effort.

This paper makes the following contributions:

1. We propose CAPs as an extension of the annotation-based DSL DCSL. Each CAP comprises a *Class Diagram Structure*, a *Parameterized OCL Template*, and an *Annotation Specification*, from which concrete OCL invariants are automatically derived through a deterministic generation mapping. This mapping provides the formal foundation for trustworthy AI-assisted constraint specification, in which AI-suggested CAP instantiations are validated before OCL derivation.
2. We develop CAP/DCSL, a supporting tool extending the UML-based Specification Environment (USE) with CAP specification, automatic OCL deriva-

tion, UDM composition, and model verification, providing a trustworthy execution pipeline for AI-assisted prototype generation.

The remainder of this paper is organized as follows. Section 2 introduces the motivating example. Section 3 presents the overall approach. Sections 4 and 5 describe the proposed method and tool support. Section 6 reviews related work, and the final section concludes the paper.

2 Motivating Example

This section presents a motivating example that illustrates the problem addressed in this paper: how complex OCL constraints can be embedded in executable domain models.

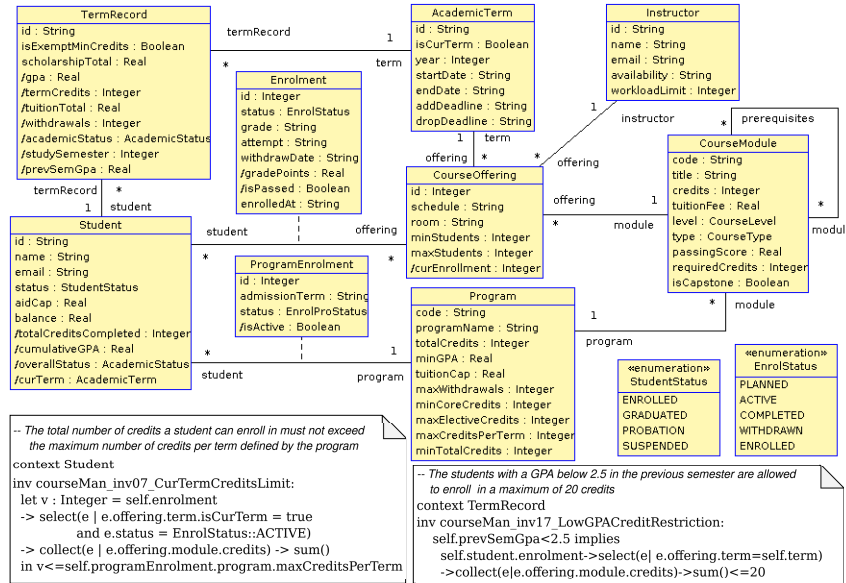


Fig. 1. The **CourseMan** domain model represented as a UML/OCL class diagram.

The motivating example is a course management system (**CourseMan**), whose domain model is shown in Fig. 1. The model includes the core entities **Student**, **Program**, **CourseModule**, **CourseOffering**, **Enrolment**, and **TermRecord**, together with their structural relationships. While the class diagram captures the structural aspects of the domain, business rules are specified using OCL constraints. For example, the constraint *inv07* in Fig. 1 requires that the total credits of all course offerings in which a student enrolls during an academic term must not exceed the maximum credit limit defined by the student’s program.

For a domain model to be executable, such constraints must be embedded in the implementation and enforced at runtime. To this end, in previ-

ous work [20] introduced DCSL, an annotation-based DSL that represents domain models directly in Java. DCSL organizes domain models around three core concepts—`DomainModel`, `Concern`, and `Annotation`—and provides structural annotations such as `DClass`, `DAttr`, `DAssoc`, and `DOpt`. For example, in `CourseMan`, `@DAttr(optional=false, length=30)` specifies that the name attribute is mandatory with a maximum length of 30. In this way, DCSL embeds essential structural constraints—object and field properties and multiplicity constraints—into the executable implementation [20, 22].

However, constraints such as `inv07` lie beyond this fixed vocabulary: they combine association navigation, collection filtering, aggregation, and conditional comparison. Lacking a declarative representation at the annotation level, such recurring constraints must be implemented as handwritten validation code—error-prone, scattered, and disconnected from the domain model. The challenge is to represent them declaratively within the annotation-based domain model while preserving their OCL semantics and automatic enforcement.

3 Overview of Our Approach

Figure 2 provides an overview of our approach, which consists of two main phases:

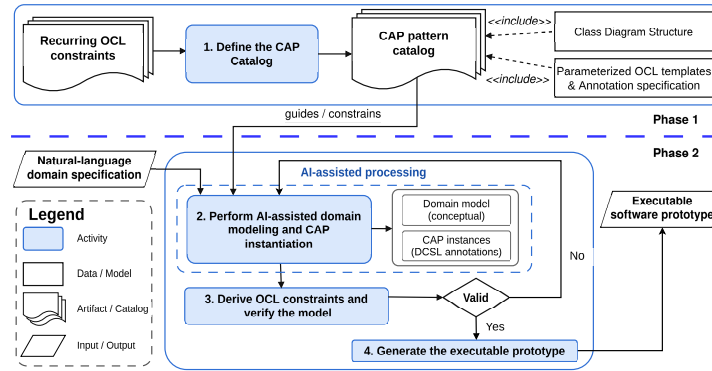


Fig. 2. Overview of the proposed approach.

Phase 1 defines a CAP catalog from recurring OCL constraints. *Phase 2* uses the catalog to guide AI-assisted domain modeling and constraint specification from a natural-language domain specification, followed by model validation and automatic prototype generation. The main steps are:

Step 1 - Define the CAP catalog. Analyze recurring OCL constraints across domains and generalize them into reusable CAPs, each specified by its template variants (Section 4.1).

Step 2 - Specify the domain model with AI-assisted CAP instantiation. Analyze the natural-language domain specification to derive a domain model, identify applicable CAPs, and instantiate them as DCSL annotations. The CAP

catalog guides and constrains AI suggestions by restricting them to valid CAP instantiations.

Step 3 - Derive OCL invariants and validate the UDM. Derive OCL invariants from CAP instantiations through the generation mapping, integrate them into the UDM, and validate the result using the USE OCL engine together with domain-expert review. If validation fails, refine the domain model and CAP instantiations before repeating this step.

Step 4 - Generate the executable prototype. Automatically generate an executable software prototype from the validated UDM.

4 Incorporation of CAPs into Domain Models

This section presents the proposed CAP method, including the DCSL extension and an initial CAP catalog for representing complex OCL constraints in UDM.

4.1 Syntax Aspect

The basic idea of a CAP is to represent recurring OCL constraints using parameterized templates. Each CAP captures a family of structurally related OCL constraints, where each structural form is represented by a *template variant* (Definition 1). Applying a CAP consists of selecting a template variant and assigning values to its annotation parameters. Rather than following the syntax of OCL, CAP parameters are defined according to the semantics of the constraint within the domain model, making annotations more intuitive while preserving the meaning of the original constraint. To support this mechanism, we extend DCSL with CAP specifications. Each CAP is described in the catalog by the following elements:

Pattern Name: A unique identifier of the CAP, facilitating its management and retrieval from the CAP catalog.

Description: A brief explanation of the purpose and semantics of the family of OCL constraints represented by the CAP.

Template Variants: A set of parameterized templates, each capturing one structural form of the constraint family. Each template variant comprises three complementary elements:

- (i) *Class Diagram Structure* — the relevant fragment of the class diagram that defines the structural context of the variant, declaring the classes, attributes, and associations to which the template parameters refer.
- (ii) *Parameterized OCL Template* — the parameterized OCL invariant expression that defines the semantic structure of the constraint, conforming to the OCL syntax.
- (iii) *Annotation Specification* — the annotation-based structure that declares the parameters of the variant and provides the mechanism for binding domain-model elements to them.

Example: A concrete illustration of how the CAP is applied by instantiating one of its template variants.

In summary, a CAP provides a reusable, parameterized mechanism for representing a family of recurring OCL constraints within domain models. Definition 1 formalizes this notion:

Definition 1 (Constraint Annotation Pattern).

A *Constraint Annotation Pattern (CAP)* is a reusable, parameterized specification for a family of structurally similar OCL constraints. Formally, a CAP is defined as the tuple $CAP = (N, D, \mathcal{V})$, where N is the pattern name, D is its description, and $\mathcal{V} = \{V_1, \dots, V_n\}$ ($n \geq 1$) is a set of template variants. Each template variant $V_i = (S_i, T_i, A_i)$ consists of: (i) a class diagram structure S_i defining the applicable structural context, (ii) a parameterized OCL template T_i representing the reusable constraint semantics, and (iii) an annotation specification A_i defining the parameter set P_i used to instantiate T_i .

A CAP is instantiated by selecting a template variant V_i and binding concrete domain-model elements or values to the parameters in P_i through A_i . Substituting these bindings into T_i derives a concrete OCL invariant, as formalized by the generation mapping in Definition 2.

Note that the *Example* serves as illustrative documentation within the catalog entry; only the *Pattern Name*, *Description*, and *Template Variants* constitute the formal structure of a CAP, as formalized in Definition 1.

4.2 An Example and the First CAP Catalog

In this section, we introduce a CAP pattern named **SumConstraint** to illustrate our proposed approach. We then describe the first catalog of CAP that we have collected and specified from various domain sources.

Pattern Name: SumConstraint

Description: This pattern defines OCL constraints for controlling and managing quantities, values, or resources through aggregation operations. Specifically, this pattern ensures that the aggregate value stored in an entity always equals the total of its related components. It is used to prevent discrepancies between derived data and source data, thereby supporting reconciliation and auditing.

Template Variant (Type01): This variant is specified through the following three elements:

- *Class Diagram Structure:* Figure 3 (label I) shows three classes, their attributes, and associations, defining the structural context of the variant.
- *Parameterized OCL Template:* Figure 3 (label II) shows the OCL template of this variant: the values of `sumAttr` over the `ClassD` objects associated with the current `ClassA` instance, filtered by `filterAttr`, are summed and validated against `thresholdAttr`. Template variables begin and end with “`_`”; the name `sumConstraint01_invID` indicates template variant `Type01` of the `SumConstraint` pattern.
- *Annotation Specification:* We extend DCSL with new annotations Figure 3 (label III) to declare the template parameters; assigning them concrete values enables the derivation of the OCL constraint.

Example: We focus on the **CourseMan** model, as shown in Fig. 1, to illustrate this pattern. The underlying constraint ensures that the maximum number of credits a student can register for is limited to 12 credits per semester when the student is under academic warning. This helps the student focus on improving their GPA. The constraint is expressed in OCL as follows:

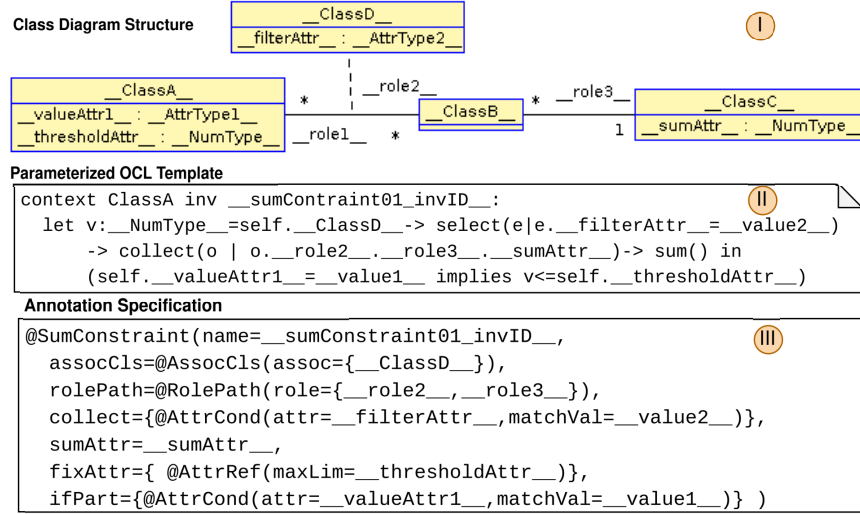


Fig. 3. Template variant (Type01) of the CAP SumConstraint.

```

context Student inv courseMan_inv06_ProbationRules :
  let v : Integer = self.enrolment
  -> select (e | e.status = EnrolStatus::ACTIVE)
  -> collect (e | e.offering.module.credits) -> sum()
  in self.overallStatus = AcademicStatus::PROBATION
  implies v <= 12
  
```

We define the specification in DCSL to derived the constraint as follows:

```

@SumConstraint(name='courseMan_inv06_ProbationRules',
  assocCls=@AssocCls(assoc={'Enrolment'}),
  rolePath=@RolePath(role={'offering', 'module'}),
  collect={@AttrCond(attr='status',
    matchVal='EnrolStatus::ACTIVE')},
  sumAttr='credits', fixAttr={@AttrRef(maxLim = 12)},
  ifPart={@AttrCond(attr='overallStatus',
    matchVal='AcademicStatus::PROBATION')}
)
class Student {...}
  
```

In this example, `rolePath` navigates to the aggregated `credits`, while `rolePath2` denotes a secondary navigation chain, required by certain `SumConstraint` template variants, that binds an auxiliary context object—here, the term of the `TermRecord`—so that only enrolments whose `offering.term` matches it are selected.

To accommodate different structural forms of the same constraint family, a CAP may define multiple template variants (Definition 1) that share the same semantics but differ in their OCL structure; for example, **SumConstraint** includes several aggregation-based variants, of which Fig. 3 illustrates one. During modeling, the user selects the appropriate template variant and binds its parameters through the corresponding *Annotation Specification*. The resulting OCL invariant is then automatically derived using the generation mapping.

Although these template variants share the same pattern name, they differ in their structural formulation. During modeling, the appropriate template variant is selected, and its parameters are instantiated via a DCSL annotation.

Users formulate constraints by instantiating the corresponding CAP annotation with appropriate parameter values. The concrete OCL invariant is then automatically generated from the selected template variant.

The **SumConstraint** pattern contains several OCL template variants corresponding to different groups of OCL constraints. The OCL template illustrated in Fig. 3 is one such variant. For example, consider another OCL constraint on the domain model shown in Fig. 1, as described below. This constraint ensures that students with a GPA below 2.5 in the previous semester are allowed to enroll in a maximum of 20 credits.

```
context TermRecord
inv courseMan_inv17_LowGPACreditRestriction :
    self.prevSemGpa < 2.5 implies
        self.student.enrolment
            ->select(e | e.offering.term = self.term)
            ->collect(e | e.offering.module.credits) ->sum() <= 20
```

The OCL template of the **SumConstraint** pattern to generate this constraint is defined as follows:

```
@SumConstraint(
    name='courseMan_inv17_LowGPACreditRestriction',
    assocCls=@AssocCls(assoc={'Enrollments'}),
    rolePath=@RolePath(role={'student','offering','module'}),
    rolePath2=@RolePath(role={'term','termRecord'}),
    collect={@AttrCond(attr='curTerm',matchVal='true')},
    sumAttr='credits',
    fixAttr={@AttrRef(maxLim=20)},
    ifPart={@AttrCond(attr='prevSemGpa',maxLim=2.5)}
)
class TermRecord { ... }
```

In this example, **rolePath** reaches the aggregated **credits**, while **rolePath2** provides the term reference that enables selecting only enrolments whose **offering.term** corresponds to the record's term. The **rolePath2** parameter denotes a secondary navigation chain required by certain **SumConstraint** template variants to bind an auxiliary context object (e.g., the current term of **TermRecord**) used for matching or filtering.

Although this constraint differs syntactically from the template shown in Fig. 1, it belongs to the same semantic family of conditional aggregation constraints. Therefore, it can be represented as another template variant within the **SumConstraint** pattern.

We identified a CAP catalog that includes ten patterns, each formally specified and managed within a unified modeling framework. Each CAP represents a family that captures the shared semantic intent of constraints of the same kind, generalizing multiple domain rules with similar structures. Each CAP includes a set of types that distinguish between different OCL template variants of the CAP pattern. The two example OCL invariants discussed in this section are generated by the two OCL templates of the **SumConstraint** pattern.

4.3 Semantics Aspect

CAP templates define parameterized OCL invariants, while DCSL annotations provide the parameter assignments required to instantiate them within a domain model. We distinguish two complementary mapping mechanisms: *direct annotation mapping*, which applies to essential structural constraints represented directly by DCSL annotations [20], and *template-based instantiation*, which applies to recurring complex OCL constraints represented as CAP template variants. In the latter case, structural elements of an OCL invariant (e.g., navigation expressions, filtering conditions, aggregation operators, and comparison expressions) are captured as annotation parameters according to the parameter-to-OCL mapping rules, which define, for each parameter kind, the OCL fragment it produces (e.g., a role path yields a navigation chain, a filtering condition yields a **select** expression, and an aggregation attribute yields a **collect-sum** expression); the complete rule set is specified in the accompanying technical report ³.

Definition 2 (CAP Application). *Let CD be a class diagram, OCL_{CD} the set of valid OCL invariants over CD , $CAP_i = (N_i, D_i, V_i)$ a CAP (Definition 1), and $V_{ij} = (S_{ij}, T_{ij}, A_{ij}) \in V_i$ a template variant whose parameter set P_{ij} is declared by the annotation specification A_{ij} . Let $\mathcal{D}_{ij}$ denote the set of well-formed parameter assignments $d : P_{ij} \rightarrow \text{Elem}(CD)$ that conform to the class diagram structure S_{ij} . A CAP application consists of selecting a template variant V_{ij} and binding a parameter assignment $d \in \mathcal{D}_{ij}$ through A_{ij} . The resulting OCL invariant is obtained by substituting each template parameter with its assigned value. Formally, $\mathcal{G} : (CAP_i, V_{ij}, d) \mapsto oclInv \in OCL_{CD}$, where $\mathcal{G}(CAP_i, V_{ij}, d)$ derives a concrete OCL invariant by replacing every parameter $p \in P_{ij}$ in T_{ij} with $d(p)$, according to the parameter-to-OCL mapping rules summarized in Table 1. Since d is well-formed, the derived OCL invariant is guaranteed to be syntactically and structurally valid over CD .*

Example. The OCL invariant `courseMan_inv06_ProbationRules` is obtained by applying the **SumConstraint** CAP to the **CourseMan** domain model. Let V_{ij} denote the corresponding template variant and let $d \in \mathcal{D}_{ij}$ be the parameter assignment specified by the **@SumConstraint** annotation, including the navigation path (**rolePath**), filtering condition (**collect**), aggregation attribute (**sumAttr**), conditional guard (**ifPart**), and comparison threshold (**fixAttr**).

Table 1. Parameter-to-OCL mapping rules of the generation mapping $\mathcal{G}$. Each annotation parameter of A_{ij} is substituted into the corresponding placeholder of the OCL template T_{ij} .

Annotation parameter	OCL fragment produced	Role in the invariant
<code>assocCls = {C}</code>	Context navigation source <code>self.c</code> (association end of C).	Collection under constraint
<code>rolePath = {r₁, ..., r_n}</code>	Navigation chain <code>.r₁.r_n.</code>	Navigation expression
<code>rolePath2 = {q₁, ..., q_m}</code>	Auxiliary navigation chain <code>.q₁.q_m</code> binding a context object.	Auxiliary context binding
<code>collect = @AttrCond(a, v)</code>	<code>->select(e e.a = v)</code>	Filtering condition
<code>sumAttr = a</code>	<code>->collect(o o.a)->sum()</code>	Aggregation operator
<code>fixAttr = @AttrRef(maxLim=k)</code>	<code>v <= k</code>	Comparison threshold
<code>ifPart = @AttrCond(a, v)</code>	<code>self.a = v implies ...</code>	Conditional guard

Applying the generation mapping $\mathcal{G}(CAP_i, V_{ij}, d)$ derives the concrete OCL invariant `courseMan_inv06_ProbationRules` presented in Subsection 4.2.

5 Tool Support and Evaluation

In this subsection, we present the methodology for developing the supporting tool, called CAP/DCSL. The tool is implemented as an extension of the UML-based Specification Environment (USE) [7], augmenting its native support for UML structural modeling and OCL constraint validation with annotation-based CAP specification and automatic invariant reconstruction.

To enable the composition of CAPs into a UDM, CAP/DCSL extends the USE grammar and processing pipeline to (i) recognize CAP annotations, (ii) associate them with cataloged CAP templates, and (iii) automatically generate corresponding OCL invariants according to the formal generation mapping defined in Section 4.3. The generated invariants are then validated using the underlying USE OCL engine to ensure syntactic and structural consistency with the domain model.

The implementation is available as an open-source project on GitHub¹.

Specifically, Fig. 4 provides an architectural overview of the CAP/DCSL tool, illustrating the extension layers built on top of USE and the workflow from annotation parsing to OCL reconstruction and UDM composition.

At the top-left (label (1)) of Fig. 4, the interface shows the four main functions of the CAP/DCSL tool, including: (i) Create CAP, (ii) Convert Standalone

¹ <https://github.com/authorsCAP/toolCAP>

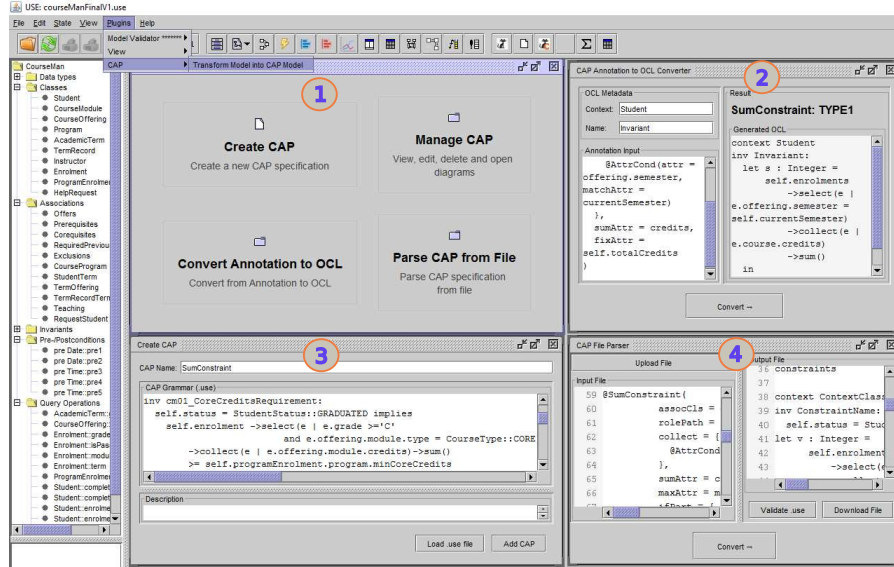


Fig. 4. CAP/DCSL supporting tool for CAP management and OCL reconstruction

Annotation to CAP, (iii) Parse Annotations from File (.use context), and (iv) Manage CAP: Each CAP may contain multiple types. This function allows users to define and manage these types within a predefined CAP.

At the top-right (label (2)) of Fig. 4, the interface presents the **Convert Standalone Annotation to OCL** function. This function focuses on parsing standalone annotations, i.e., annotations that are not associated with a specific model context. To transform such annotations into OCL constraints, the user must manually define the context (e.g., the class in which the annotation should be applied, such as `Student` or `CourseModule`) and provide an invariant name, which will serve as the identifier of the generated OCL constraint.

At the bottom-left (label (3)) of Fig. 4, the interface shows the **Create CAP** function. This function allows users to define a new CAP type. When creating a CAP, the user specifies the CAP name (e.g., `SumConstraint`) and provides a corresponding ".use" file that illustrates the class diagram structure associated with that CAP. The system allows the ".use" file to be validated to ensure that its grammar and syntax conform to the USE specification.

At the bottom-right (label (4)) of Fig. 4, the interface shows the **Parse Annotations from a ".use" File** function. This function processes annotations that already appear within a ".use" model file. When such a file is loaded, the plugin analyzes the annotations together with their model context, reconstructs the corresponding OCL constraints, and generates a new ".use" file in which the annotations are automatically converted into these constraints. The reconstructed constraints are then integrated into the UDM. After generation, the

resulting ".use" file can be validated to ensure that its grammar and syntax are correct.

Evaluation and Discussion. To substantiate the evaluation results and minimize subjective interpretation, we conducted controlled reconstruction experiments comparing CAP with four platforms: DCSL [20], Apache Causeway [10], OpenXava [26], and Actifsource [1].

Case studies. For each platform, we reconstructed functionally equivalent domain models for three case studies—**CourseMan**, **ProcessMan**, and **OrderMan**—selected to ensure domain diversity, constraint diversity, and varying modeling maturity. **CourseMan** is a benchmark academic domain with rich structural and behavioral constraints. **ProcessMan** enables controlled comparison with prior DCSL-based modeling to assess incremental benefits of CAP. **OrderMan**, adapted from enterprise business-process scenarios, represents a heterogeneous, workflow-intensive domain to evaluate scalability and general applicability.

Table 2. Summary of case studies: **CourseMan**, **ProcessMan**, and **OrderMan**

Case Study	Domain Type	OCL Constraints	Semantic OCL Constraint Types	Role in Evaluation
CourseMan	Academic	184	14	Expressiveness benchmark
ProcessMan	Process management	140	12	Controlled DCSL comparison
OrderMan	Business workflow	76	11	Scalability & generality

Table 2 summarizes the quantitative characteristics of the three systems, including the number of constraints and the semantic OCL Constraint Types used in the evaluation.

Controlled reconstruction protocol. For each case study and each platform, we performed the following steps:

1. Reconstruct the domain model (class diagram) using the platform’s native mechanisms;
2. Encode the constraints to match the reference specification as closely as possible;
3. Record measurable indicators:
 - number of constraints supported directly by annotations or built-in constructs,
 - number requiring manual implementation,
 - manual validation code size (LOC) for each constraint,

- implementation time,
 - availability of automatic generation support (model-to-code and validation generation).
4. Validate the reconstruction results independently by two researchers; discrepancies were resolved by re-checking the encoding artifacts and reaching consensus.

All evaluation dimensions reported in the following subsections are grounded in these recorded artifacts and measurements. We focus on expressiveness and required coding effort.

Expressiveness of CAP in Domain Modeling Expressiveness evaluates the ability of each platform to specify and enforce domain constraints declaratively at the model level. We operationalize expressiveness using twelve criteria derived from recurring constraint families across the three case studies.

Table 3. Comparison of platforms by expressiveness levels (12 criteria)

Platform	Fully supported	Partially supported	Not supported
CAP	10/12	2/12	0/12
DCSL	4/12	5/12	3/12
OpenXava	2/12	7/12	3/12
Apache Causeway	2/12	7/12	3/12
Actifsource	9/12	3/12	0/12

Results. Table 3 summarizes how many of the twelve criteria fall into each level for each platform.

Under the 12 expressiveness criteria, CAP supports all categories (12/12), comprising 10 fully supported and 2 partially supported criteria, with no unsupported cases. DCSL, OpenXava, and Apache Causeway each leave three criteria unsupported (3/12). Actifsource also covers all criteria (12/12), though several criteria are supported via meta-model configuration and model-level rules rather than direct annotation-based constructs.

Required Coding Effort

This evaluation focuses exclusively on the *essential structural OCL constraints* defined in Table 1 (Subsection 2.3), which summarizes the set of core domain constraints considered in the study. The objective is to quantify the handwritten implementation effort required to enforce these constraints when using the CAP-extended DCSL compared to other platforms.

Evaluation Scope. For each of the three case studies (**CourseMan**, **ProcessMan**, and **OrderMan**), we reconstructed identical domain models and implemented the same set of essential well-formedness OCL constraint categories in DCSL [20], including:

- attribute value constraints,

- multiplicity and cardinality constraints,
- dependency constraints,
- aggregation-based constraints (e.g., sum/count),
- cross-association structural invariants.

Only essential constraints were included in the measurement, as they represent the core domain integrity rules targeted by CAP. Auxiliary and framework-level constraints were excluded because they are typically handled automatically and would therefore not provide a meaningful basis for evaluating the reduction in manual modeling effort.

Measurement Protocol. For each platform and each essential constraint instance, we recorded:

1. whether the constraint could be enforced declaratively without handwritten validation code;
2. the number of handwritten validation methods required;
3. the number of handwritten validation LOC (excluding comments and blank lines);
4. the number of auxiliary validator/service classes introduced solely for enforcement.

Note that, structural class declarations were not counted as validation code because they belong to the domain model specification rather than to constraint validation logic, which is the focus of CAP.

Table 4. Required coding effort for essential OCL constraints

Platform	Avg manual LOC/constraint	% auto-enforced essential constraints	Additional validator classes
CAP-extended DCSL	6.2	92%	0
Apache Causeway	54.3	22%	6
OpenXava	37.5	45%	4
Actifsource	31.8	52%	3

Quantitative Results. Table 4 summarizes the aggregated results averaged over the three case studies.

CAP as a Foundation for Trustworthy AI-Assisted Prototype Generation. LLMs can assist UML/OCL specification, but their outputs require formal verification before entering model-driven software generation [18, 8, 2]. CAP provides such a verification layer by restricting AI suggestions to CAP instantiations, which are structurally validated, from which OCL invariants are deterministically derived by the generation mapping $\mathcal{G}$, checked by the USE OCL engine, and reviewed by domain experts before integration into the UDM. This establishes the foundation

for trustworthy AI-assisted prototype generation, while empirical evaluation of LLM-assisted CAP recommendation is left for future work.

The current CAP catalog focuses on invariant-based constraints and excludes operational concerns such as pre/postconditions, security policies, and event-driven behaviors. Validity threats were mitigated through explicit modeling, evaluation on three heterogeneous case studies, and tool-based validation, although further studies on additional DDD applications are needed to strengthen generalizability.

6 Related Work

Several studies have translated UML/OCL specifications into executable code or annotations in object-oriented languages. Existing approaches generate Java validation code [27], translate OCL into Java annotations [16], or combine OCL with annotations for domain modeling and verification [5, 9, 12]. However, most approaches support only limited subsets of OCL, require manual intervention for complex constraints, or focus on constraint validation rather than reusable domain-level specification. DDD [13] places the problem domain at the center of software development, and several studies [30, 25, 19, 4, 15] apply it to construct rich domain models that capture business rules, workflows, and constraints while supporting model-driven generation of software artifacts [28, 21]. A fundamental objective is to preserve consistency between domain models and implementations so that the models evolve together with business requirements. Recent studies [8, 2] have applied LLMs to software modeling and OCL specification tasks. These approaches generate free-form artifacts that require manual correction, whereas CAP provides a verifiable intermediate layer: AI suggestions are restricted to pattern instantiations and are formally checked before OCL derivation. DCSL [20] introduced an annotation-based internal DSL for specifying structural domain models and essential OCL constraints. This paper extends DCSL Patterns (CAPs) with constraint annotation, which provide reusable parameterized templates for recurring complex OCL constraints, together with a formal generation mapping that derives OCL invariants and composes them into the UDM for executable software generation.

7 Conclusion and Future Work

This paper presented *Constraint Annotation Patterns* (CAPs), a pattern-based extension of an annotation-based DSL for representing recurring complex OCL constraints in DDD. CAPs enable reusable constraint specification, automatic derivation of OCL invariants through a deterministic generation mapping, and their integration into the UDM for model verification and prototype generation. The approach is implemented in the CAP/DCSL tool as an extension of USE. Evaluation on three heterogeneous case studies demonstrates that CAPs improve the expressiveness of annotation-based domain models while reducing manual validation effort through reusable constraint specifications.

Future work will extend the CAP catalog to support additional constraint families and investigate LLM-assisted CAP recommendation for trustworthy AI-assisted prototype generation, where AI-generated constraint specifications are

validated against CAP templates and derived into verifiable OCL invariants before entering the software generation pipeline.

References

1. Actifsource. Actifsource AG, Switzerland - all rights reserved. (2017)
2. Abukhalaf, S., Hamdaqa, M., Khomh, F.: On codex prompt engineering for ocl generation: an empirical study. In: 2023 IEEE/ACM 20th Int. Conf. Mining Software Repositories (MSR). pp. 148–157. IEEE (2023)
3. Andrzej Wąsowski, Thorsten Berger: Domain-Specific Languages Effective Modeling, Automation, and Reuse. Springer Cham (2023)
4. Brambilla, M., Cabot, J., Wimmer, M.: Model-Driven Software Engineering in Practice. Morgan&Claypool, 2nd edn. (2017)
5. Brucker, A.D., Krieger, M.P., Longuet, D., Wolff, B.: A Specification-Based Test Case Generation Method for UML/OCL. In: Models in Software Engineering. pp. 334–348. Springer, Berlin, Heidelberg (2011), iSSN: 1611-3349
6. Bucchiarone, A., Cicchetti, A., Ciccozzi, F., Pierantonio, A.: Domain-specific languages in practice: with JetBrains MPS. Springer Nature (2021)
7. Büttner, F., Gogolla, M.: On ocl-based imperative model transformation. Electronic Communications of the EASST **29** (2010)
8. Cámara, J., Troya, J., Burgueño, L., Vallecillo, A.: On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. Software and Systems Modeling **22**(3), 781–793 (2023). <https://doi.org/10.1007/s10270-023-01105-5>
9. Córdoba-Sánchez, I., De Lara, J.: Ann: A domain-specific language for the effective design and validation of Java annotations. Computer Languages, Systems Structures **45**, 164–190 (Apr 2016)
10. Dan Haywood: Apache Isis - Developing Domain-Driven Java Apps. Methods & Tools: Practical knowledge source for software development professionals **21**(2), 40–59 (2013)
11. Dang, D.H., Le, D.M., Le, V.V.: AGL: Incorporating behavioral aspects into domain-driven design. Information and Software Technology **163**, 107284 (2023)
12. Dania, C., Clavel, M.: OCL2MSFOL: a mapping to many-sorted first-order logic for efficiently checking the satisfiability of OCL constraints. pp. 65–75. ACM (Oct 2016). <https://doi.org/10.1145/2976767.2976774>
13. Evans, E.: Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional (2004)
14. Fowler, M.: Domain-Specific Languages. Addison-Wesley Professional, 1st edn. (Sep 2010)
15. Griffin, J.: Domain-Driven Laravel, Learn to Implement Domain-Driven Design Using Laravel (Jan 2021). <https://doi.org/10.1007/978-1-4842-6023-4>
16. Hamann, L., Hofrichter, O., Gogolla, M.: OCL-Based Runtime Monitoring of Applications with Protocol State Machines. vol. 7349, pp. 384–399. Springer Berlin Heidelberg (2012). https://doi.org/10.1007/978-3-642-31491-9_29
17. Haq, F.U., Cabot, J.: B-OCL: An Object Constraint Language Interpreter in Python (2025). <https://doi.org/10.48550/arXiv.2503.00944>, arXiv:2503.00944 [cs]
18. Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology **33**(8), 1–79 (2024)

19. Jaiswal, S.K., Agrawal, R.: Domain-Driven Design (DDD)- Bridging the Gap between Business Requirements and Object-Oriented Modeling. *Int. Innovative Research in Engineering and Management* **11**(2), 79–83 (2024), number: 2
20. Le, D.M., Dang, D.H., Nguyen, V.H.: On domain driven design using annotation-based domain specific language. *Computer Languages, Systems & Structures* **54**, 199–235 (Dec 2018)
21. Le, V.V., Be, N.T., Dang, D.H.: On Automatic Generation of Executable Domain Models for Domain-Driven Design. In: 2023 15th International Conference on Knowledge and Systems Engineering (KSE). pp. 1–6 (Oct 2023). <https://doi.org/10.1109/KSE59128.2023.10299453>, iSSN: 2694-4804
22. Le, V.V., Dang, D.H.: An Approach to Composing Concerns for an Executable Unified Domain Model. In: 2024 RIVF Int. Conf. On Computing and Communication Technologies (RIVF). pp. 424–428 (Dec 2024). <https://doi.org/10.1109/RIVF64335.2024.11009109>, iSSN: 2473-0130
23. OMG: Object Constraint Language 2.4. OMG (2014)
24. OMG: Unified Modeling Language 2.5.1. Object Management Group (2017)
25. Özkan, O., Babur, Ö., van den Brand, M.: Domain-driven design in software development: A systematic literature review on implementation, challenges, and effectiveness (2023). <https://doi.org/10.48550/arXiv.2310.01905>, arXiv:2310.01905
26. Paniza, J.: Learn OpenXava by example. CreateSpace, 1.1 edn. (2011)
27. Rackov, M., Kaplar, S., Filipović, M., Milosavljević, G.: Java code generation based on ocl rules. In: 6th International Conference on Information Society and Technology. pp. 191–196 (2016)
28. Sangabriel-Alarcón, J., Ocharán-Hernández, J.O., Cortés-Verdín, K., Limón, X.: Domain-Driven Design for Microservices Architecture Systems Development: A Systematic Mapping Study. pp. 25–34. IEEE Computer Society (Nov 2023). <https://doi.org/10.1109/CONISOFT58849.2023.00014>
29. Steele, J.G.B.J.G., Buckley, G.B.A.: The Java language Specification. Bhaskarjyoti Saikia (2020)
30. Vernon, V.: Implementing Domain-Driven Design. Addison-Wesley Professional, 1 edn. (2013)
31. Völter, M., Benz, S., Dietrich, C., Engelmann, B., Helander, M., Kats, L., Visser, E., Wachsmuth, G.: DSL Engineering - Designing, implementing and using domain-specific languages. M Volter / DSLBook.org, Stuttgart, Germany (2013)